SE 2228-Analysis and Design of Algorithms

Algorithm Analysis-2

Other Asymptotic Notations

Iterative/Recursive Algorithms

Lower Bound : Ω (Big- Omega) Notation

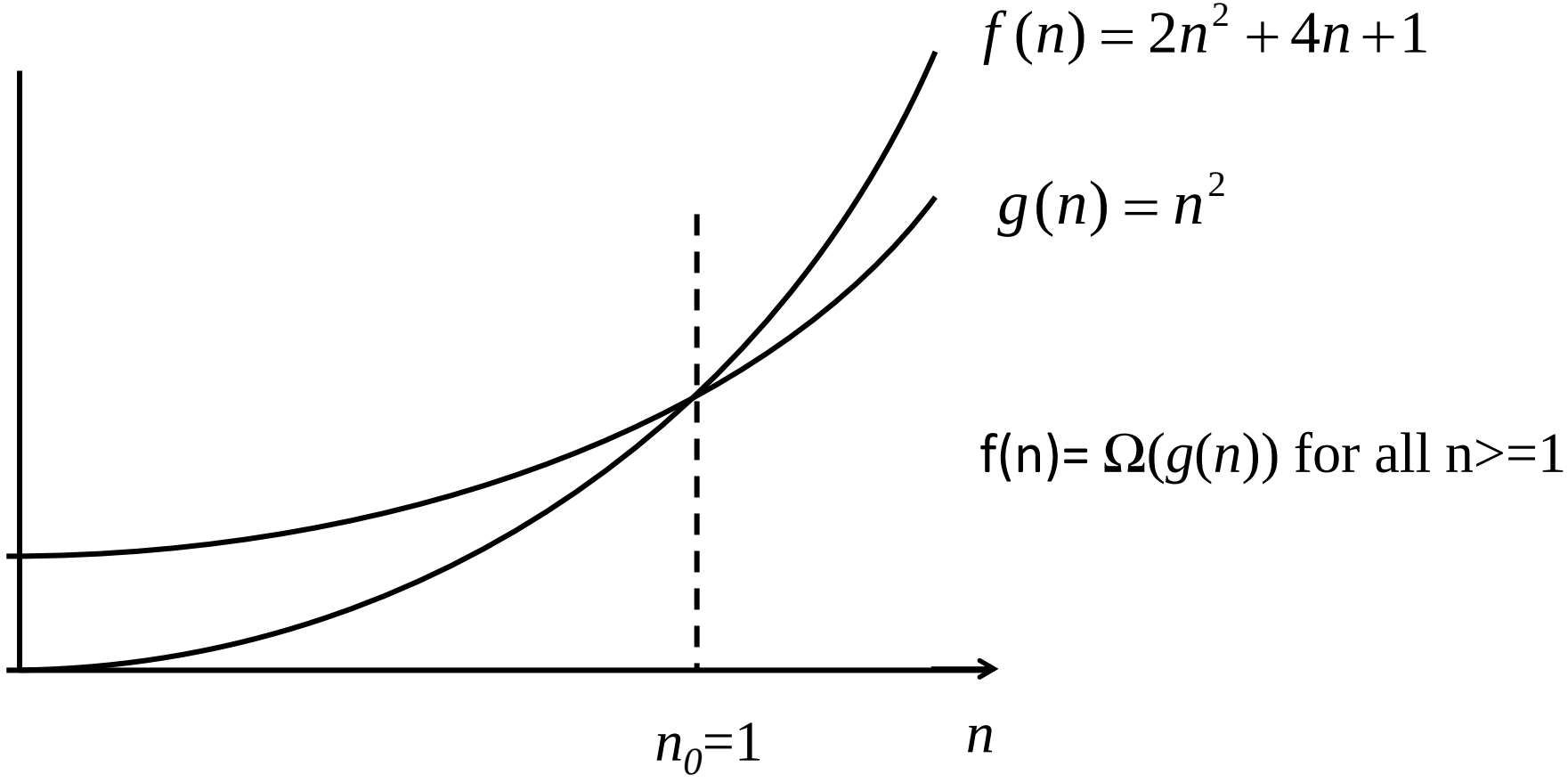
Definition:

The function $f(n) = \Omega(g(n))$ if there exists positive numbers c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

Note that the main difference from big - O is the direction of the inequality .

- ***O*-**notation provides an asymptotic upper bound on a function $f(n)$.
- Similarly Ω -notation provides an asymptotic lower bound on a function $f(n)$.

Ω - Notation: Lower Bound



Ω - Notation : Lower Bound Examples

Example 1: f(n)= n^3 is Ω (n^2) for all n>1. Because $n^3 > n^2$ for all n>1.

Example 2 :

$$f(n) = \frac{1}{2}n(n-1) = \Omega(n^2)$$

We need to show that

$$\frac{1}{2}n(n-1) = \frac{n^2}{2} - \frac{n}{2} \geq cn^2 \text{ for some values of } c > 0 \text{ and } n.$$

$$\frac{n^2}{2} - \frac{n}{2} > cn^2 \Rightarrow \frac{1}{2} - \frac{1}{2n} > c$$

Choosing $n = 2$ gives $c = 1/4$

$$\Rightarrow \text{for } n > 2 \text{ and } c \leq 1/4,\ f(n) = \Omega(n^2)$$

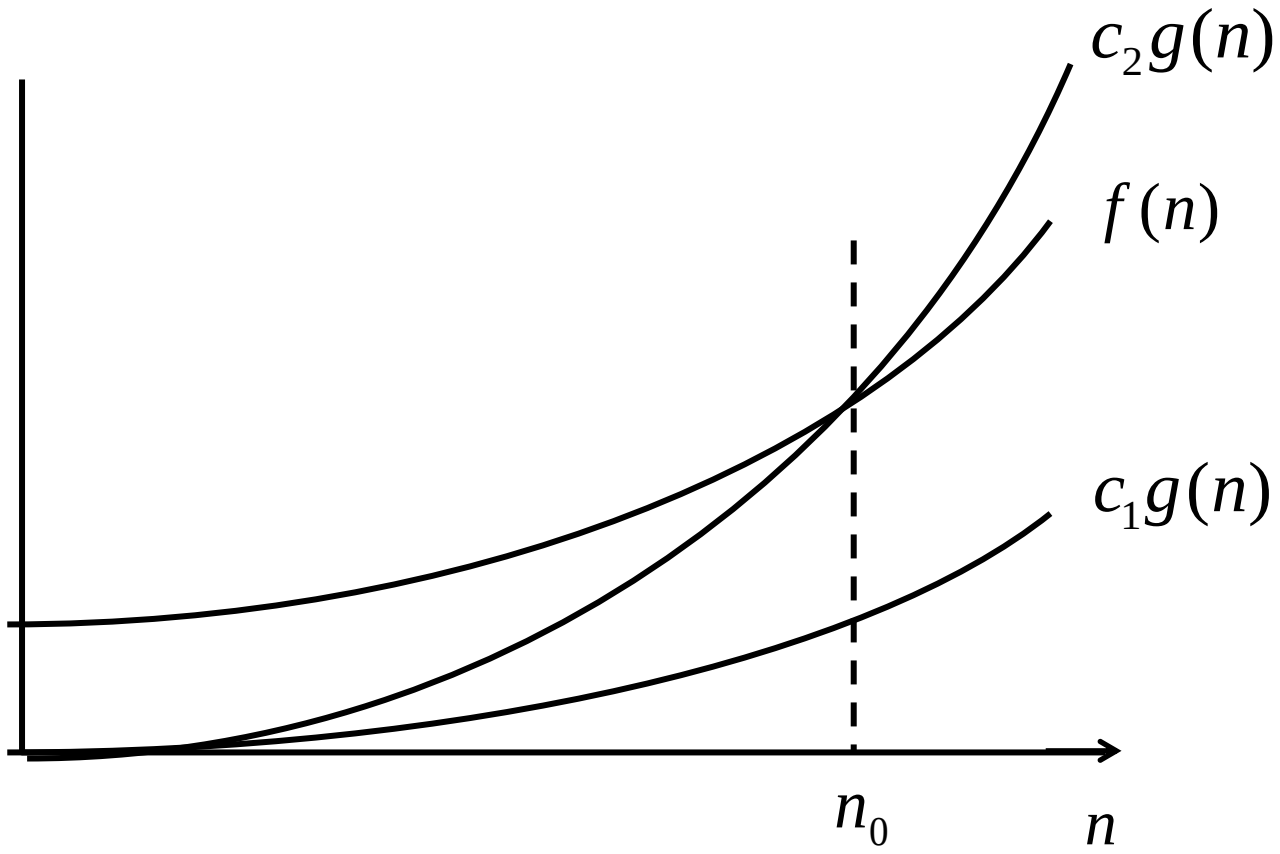
Θ -Notation (same order of growth)

The function $f(n) = \Theta(g(n))$ if there exists positive numbers c_1, c_2 and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

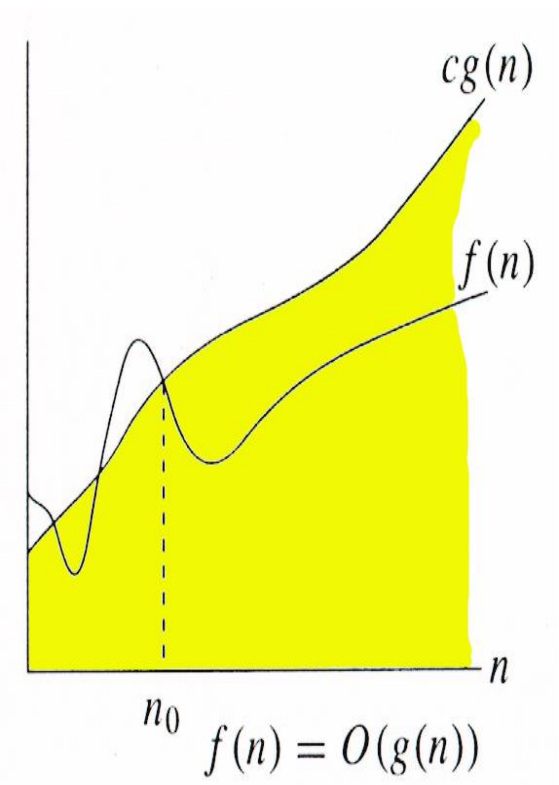
Note that $f(n)$ is $\Theta(g(n))$ if $f(n)$ is $O(g(n))$ and $f(n)$ is $\Omega(g(n))$.

Θ - Notation

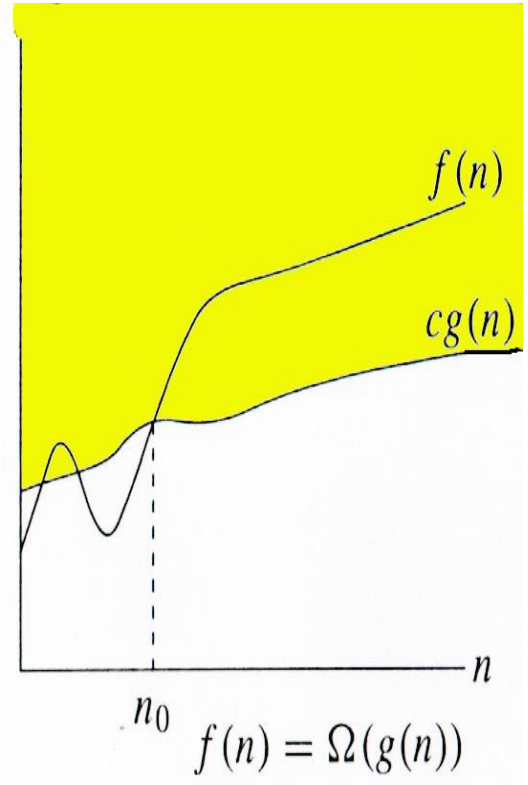
We can say that,based on this notation, f(n) and g(n) grow at the same rate <u>in the long run</u>.



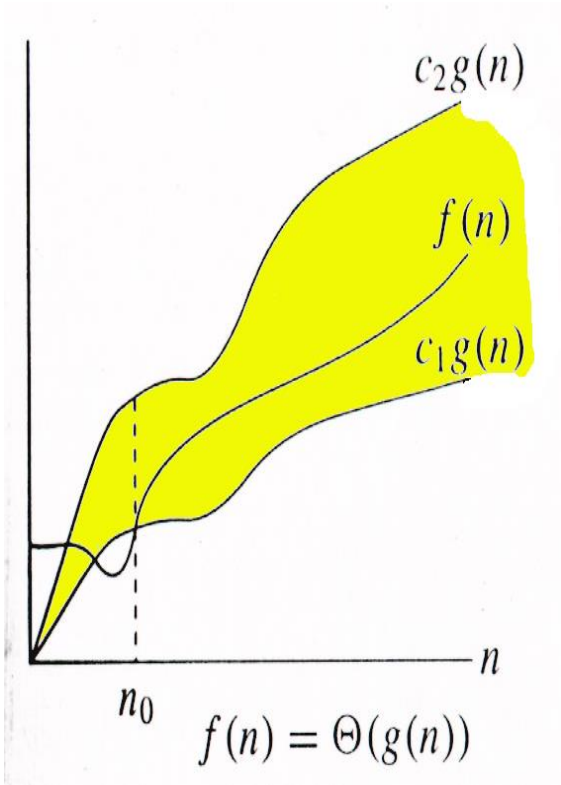
Relations Between, *O*, Ω,Θ



Upper Bound



Lower Bound



Same rate

Examples : O, Ω ,Θ

$10n^2 \in O(n^2)$

$10n^3 + 2n^2+n \in O(n^3)$

$100n + 5 \in O(n)$

$5n+20 \in O(n)$

Big-O
Upper Bound

$10n^2 \in \Omega(n^2)$

$10n^2 + 2n \in \Omega(n^2)$

$10n^3 \in \Omega(n^2)$

Ω Omega
Lower Bound

$10n^2 \in \Theta(n^2)$

$10n^2 + 2n \in \Theta(n^2)$

$(1/2)n(n-1) \in \Theta(n^2)$

Θ Teta
Same order

Classifying Growth Functions

- Given functions $f(n)$ and $g(n)$ we might like to determine if one grows faster than the other.
- To do this we take the limit of the two functions as n grows to infinity

$$\lim_{n \to \infty} \frac{f(n)}{g(n)}$$

If the limit goes to zero, then $f(n)$ is in $O(g(n))$. - Worst case

If this limit goes to ∞, then $f(n)$ is in $\Omega(g(n))$. - Best case

If the limit goes to a constant then $f(n)$ is in $\Theta(g(n))$. – The same

Rules to manipulate Big-O

- Let T1(N) and T2(N) be run times such that:

  $T1(N) = O(f(N)), T2(N) = O(g(N))$

  Then, $T1(N) + T1(N) = \max(O(f(N)), O(g(N)))$

- $T1(N) * T2(N) = O(f(N) * g(N))$

- If T(N) is a polynomial of degree **k**, then $T(N) = \Theta(N^k) = O(N^k)$.

Finding Rate of Growth ≡ Asymptotic Analysis

- Using *rate of growth* as a measure to compare different functions implies comparing them asymptotically.
- If $f(x)$ is *faster growing* than $g(x)$, then $f(x)$ always eventually becomes larger than $g(x)$ in the limit (for large enough values of x).

How to express rate of growth for different algorithm types?

We consider two categories of algorithms:

- **Iterative (nonrecursive) Algorithms**
- **Recursive Algorithms**

Iterative Algorithms

- Iterative algorithms take **one step at a time** towards the final destination (Solution).

→ Repeat operations until some condition becomes true or false.

General structure:

loop (until done)

take steps

end loop

Computing and Simplifying Running Time -1

- Method: Associate a "cost" with each statement and find the "total cost“ by finding the total number of times each statement is executed.
- Express running time in terms of the size of the problem.

Example problem:

Assign 0 to all elements of an array of size N.

→ arr[i] = 0 ; i=0,…,n-1 , or i=1,…,n

Computing and Simplifying Running Time- 2

Algorithm 1

	Cost
arr[0] = 0;	c1
arr[1] = 0;	c1
arr[2] = 0;	c1
...	
arr[n-1] = 0;	c1

T(n)= c1+c1+...+c1 = c1 x n

Algorithm 2

	Cost
for(i=0; i<n; i++)	c2
arr[i] = 0;	c1

T(n)= (n+1) x c2 + n x c1

= (c2 + c1) x n + c2

O(n)

Although the algorithms have different cost values,both are O(n) algorithms.

Computing and Simplifying Running Time -3

Example: Find the sum of the array elements :

	Cost
sum = 0;	c1
for(i=0; i<n; i++)	c2
for(j=0; j<n; j++)	c2
sum += arr[i][j];	c3

T(n) =*c*1 + *c*2 *x* (*n*+1) + *c*2 *x n x* (*n*+1) + *c*3 *x n x n*

Can be simplified to : T(n)= a n^2 + bn +c

=$O(n^2)$

Euclid's Algorithm

//Also known as Euclidean Algorithm.Non-recursive implementation)

```
long gcd ( long m, long n)
{
  long r;
  while (n != 0)
        {
              r = m % n;
              m = n;
              n  = r;
        }
   return m;
   }
```

GCD(210,45)=15

m	n	r
210	45	30
45	30	15
30	15	0
15	0	Done .

Return m=15.

Time Complexity -1

- **How many times the loop iterates. This is the same question as:**

  How many times can we apply the mod operation?
- Each iteration reduces the first argument of the mod operation:

  In the example we had : 210,45,30,15
- What can be the maximum number of iterations?
- Detailed analysis involves different possibilities . We are going to consider a simple approach.

Time Complexity -2

Consider the simple fact:

If $M > N$, then $(M \% N) < M/2$. $(r < M/2)$

→ The size of $M \bmod N$ is strictly less than $M/2$.

- So how many times % is applied for the remainder to be zero $(r = 0)$?

→The number of times mod operations is performed < the number of times M can be divided by 2

→ This can not be more than $\log_2(M)$.

→ Worst case Time complexity of Euclid's algorithm is $O(\log_2 M)$.

- Best case : O(1),If at the start the remainder is 0. Ex: (100,50)

When the worst case occurs ?

If N and M are two consecutive Fibonacci numbers (Check this!).

Example : Insertion Sort

- Take multiple passes over the list
- Keep already sorted part at low-end
- Find next unsorted element
- Insert it in correct place, relative to the ones already sorted
- Invariant: each pass increases the size of sorted portion.

Insertion Sort: How it Works ?

Insertion sort sorts the elements in place

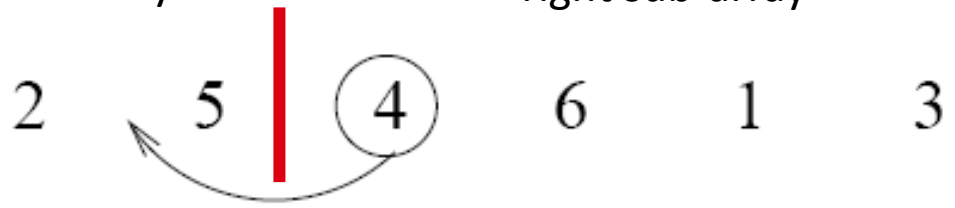
input array

A: 5 2 4 6 1 3

at each iteration, the array is divided in two sub-arrays:

left sub-array

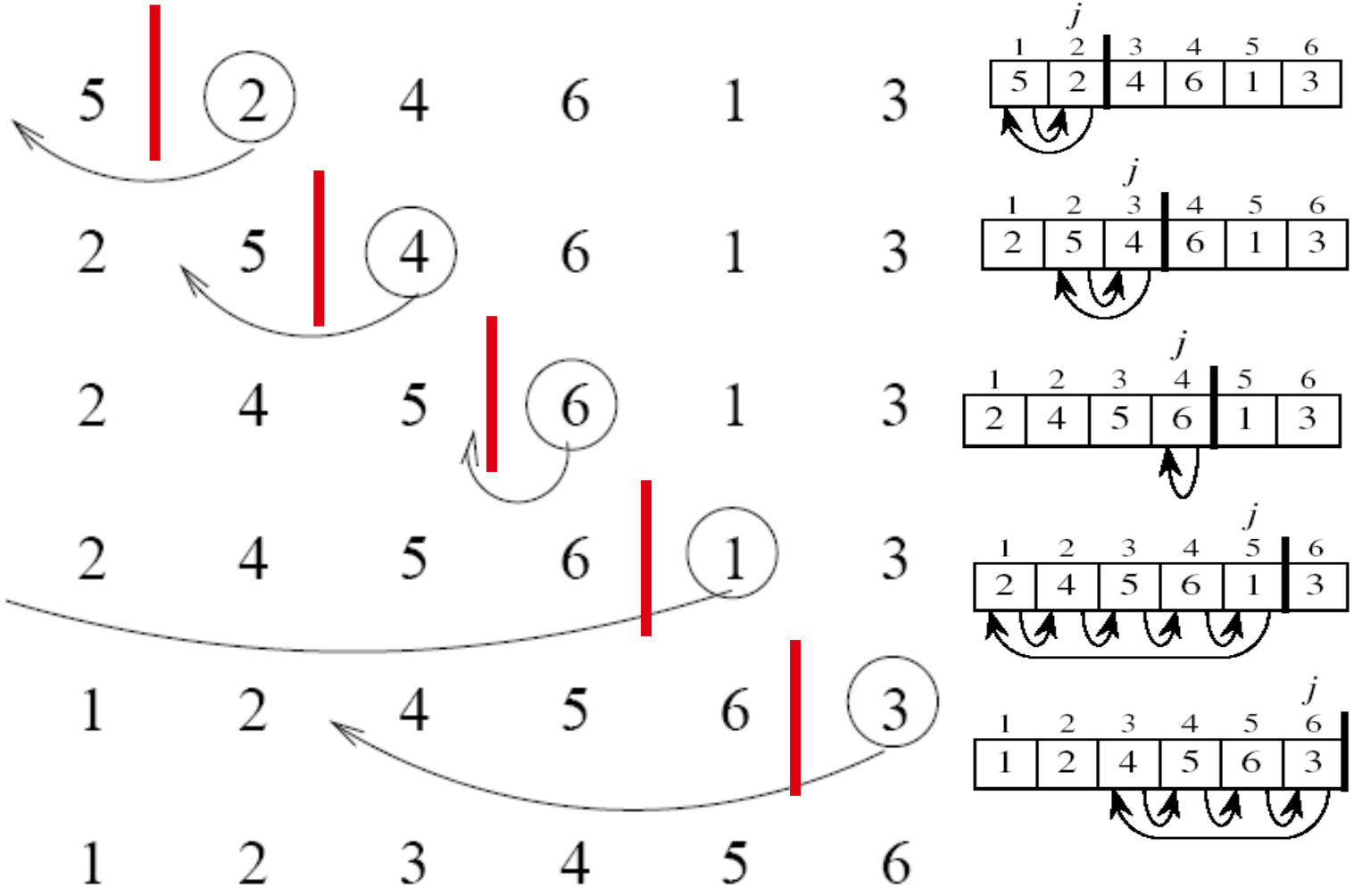
right sub-array



sorted

unsorted

Insertion Sort: How it Works ?



Reminder : INSERTION-SORT

```
INSERTION-SORT(A,n)
  for j ← 2 to n
      do key ← A[ j ]
         // Insert A[ j ] into the sorted sequence A[1 . . j -1]
         i ← j - 1
         while i > 0 and A[i] > key
             do A[i + 1] ← A[i]
                i ← i - 1
         A[i + 1] ← key
```

1	2	3	4	5	6	7	8
a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8

key

Analysis of Insertion Sort

INSERTION-SORT*(A,n)*	cost	Counts
for j ← 2 **to** n	c_1	n
do key ← A[j]	c_2	n-1
// Insert A[j] into the sorted sequence A[1 . . j -1]	0	n-1
i ← j - 1	c_4	n-1
while i > 0 and A[i] > key	c_5	$\sum_{j=2}^{n} t_j$
do A[i + 1] ← A[i]	c_6	$\sum_{j=2}^{n} (t_j - 1)$
i ← i – 1	c_7	$\sum_{j=2}^{n} (t_j - 1)$
A[i + 1] ← key	c_8	n-1

t_j: # of times the while statement is executed at iteration j

Analysis of Insertion Sort

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1)$$

What can be the value of T(n)? How many times will the inner loop be executed?

<u>Worst Case:</u>

After simplifications , the equation T(n) can be reduced to the following form by combining together linear and quadratic terms:

$T(n) = an^2 + bn + c$

where a,b,c are positive cost values.

Lower Bound

<u>Best Case:</u>

The while loop will not be executed,so no n^2 terms remain in T(n).

In that case, we can assume that :

$T(n) = an + bn$

For positive a and b we can write:

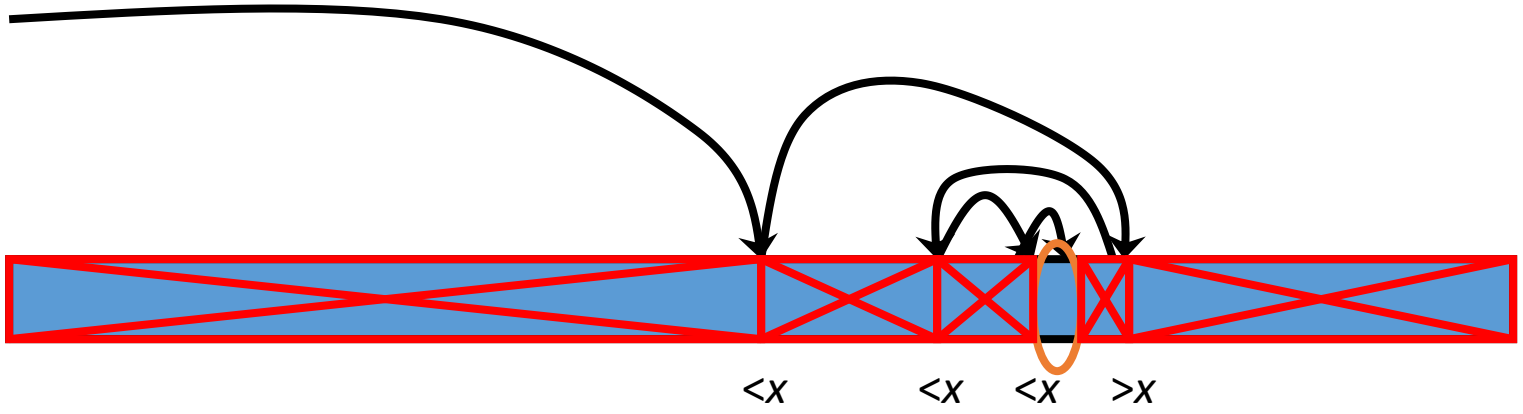
$an + b\,n \geq an$.

So Insertion sort is Ω (n)

→We say Insertion Sort's (lower bound) complexity is Ω(n)

Searching: Binary Search(Iterative Version)

- Basic idea: On each step, look at the *middle* element of the remaining list to eliminate half of it, and quickly locate the desired element.



Searching: Binary Search

```
//Input A: a1, a2, …, an: distinct integers.Search key :x
binary search(A,n,x)
   i ← 1                    //left endpoint of search interval
   j ← n                    //right endpoint of search interval
   while i<j   do
   mid ← ⌊(i+j)/2⌋          //midpoint
            if x>a_mid then
               i ← mid+1
            else
               j ← mid
         if x = a_i then      //Found
            location ← i
            else location ←0  //Not found
   return location
```

Binary Search Runtime

- Best Case: match from the first comparison: O(1)
- Worst Case: divide until reach one item, or no match
- For an array of size N, it eliminates ½ until 1 element remains.
  N, N/2, N/4, N/8, ..., 4, 2, 1
  - **How many divisions does it take?**
- **Think of it from the other direction:**
  - **How many times do I have to multiply by 2 to reach N?**
    1, 2, 4, 8, ..., N/4, N/2, N
  - **Call this number of multiplications "x".**

  $2^x = N$
  $x = \log_2 N$

- Binary search is in the **logarithmic** complexity class.

Worst-case Analysis : Recurrence

Item not in the array (size N)

$T(N)$ = number of comparisons with array elements

$T(1) = 1$

$$
\begin{aligned}
T(N) &= 1 + T(N / 2) \\
&= 1 + [1 + T(N / 4)] \\
&= 2 + T(N / 4) \\
&= 2 + [1 + T(N / 8)] \\
&= 3 + T(N / 8) \\
&= \ldots \\
&= k + T(N / 2^k) \qquad [1]
\end{aligned}
$$

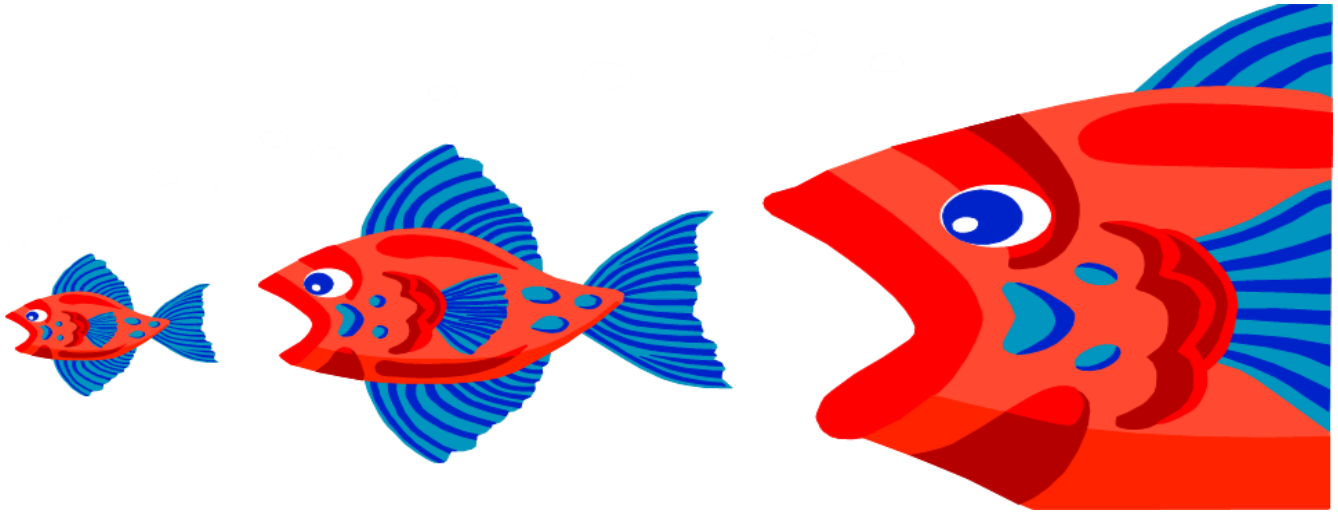
Is Binary Search more efficient?

- For a list of *N* elements, Binary Search can execute at most $\log_2 N$ times.
- A disadvantage of Binary Search is the overhead of maintaining the list in ordered form.

Number of Iterations

N	Linear Search(Av)	Binary Search(Worst)
10	5	3
100	50	6
1,000	500	9
1,000,000	500000	20

Recursive Algorithms



Recursion vs. Iteration

- Repetition
  - Iteration: explicit loop
  - Recursion: repeated function calls
- Termination
  - Iteration: loop condition fails
  - Recursion: base case recognized
- Both can have infinite loops
- Recursive solutions can be easier to understand and to describe than iterative solutions.

Example-1: Analysis of Recursive *n* !

Recursive algorithm for *n*!

Input size: *n*

Basic operation: multiplication "*"

- **Let *M*(*n*) be the number of multiplications** needed to compute *n*!, then the recurrence can be expressed as:

```
ALGORITHM Factorial(n)
if n = 0
    return 1
else
    return Factorial(n-1)*n
```

$$M(0) = 0$$

$$M(n) = M(n-1) + 1 \quad \textbf{for } n > 0$$

To compute *Factorial*(*n*-1)

To multiply *Factorial*(*n*-1) by *n*

Analysis : Solving the Recurrence

$$M(0) = 0$$

$$M(n) = M(n-1) + 1 \ \textbf{for}\ n > 0$$

$$\Rightarrow M(n)$$

$$= M(n-2) + 2$$

$$= M(n-3) + 3$$

$$= \ldots$$

$$= M(n-n) + n$$

$$= n$$

→The number of multiplications needed to compute *n*!

→ The complexity of this algorithm is O(n)

Example-2 : Recursive Fibonacci

0,1, 1, 2, 3, 5, 8, 13, 21, 34, …..

```
// Recursively calculates nth Fibonacci number
long fib (int n)
 {
   if( n <= 1 )
       return n
       else
         return fib(n – 1) + fib(n – 2);
 }
```

This is a straightforward, but inefficient recursion ...

Analysis of Recursive Fibonacci Algorithm

- Let n_k denote the **number of function calls** made by Fib(n).

  $n_0 = 1$

  $n_1 = 1$ The initial call

  $n_2 = n_1 + n_0 + 1 = 1 + 1 + 1 = \mathbf{3}$

  $n_3 = n_2 + n_1 + 1 = 3 + 1 + 1 = 5$

  $n_4 = n_3 + n_2 + 1 = 5 + 3 + 1 = \mathbf{9}$

  $n_5 = n_4 + n_3 + 1 = 9 + 5 + 1 = 15$

  $n_6 = n_5 + n_4 + 1 = 15 + 9 + 1 = 25$

  $n_7 = n_6 + n_5 + 1 = 25 + 15 + 1 = 41$

  $n_8 = n_7 + n_6 + 1 = 41 + 25 + 1 = 67.$

- Note that the value at least doubles for each two consecutive indices: n_4 is more than twice n_2 (9>2*3)….

→ **$n_k > 2^{k/2}$ [Ex: $n_8 > 2^{8/2}$]**

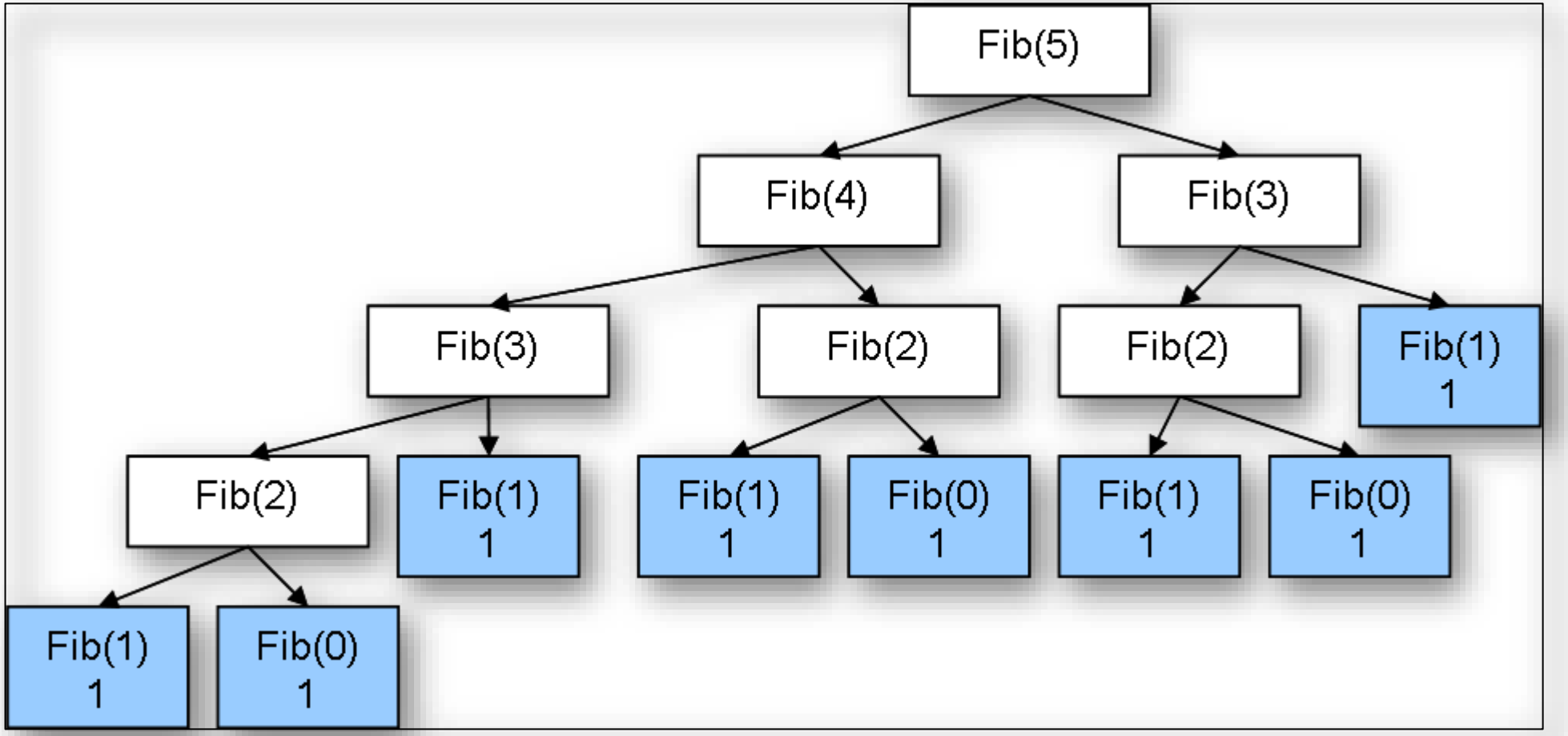
It is an exponential time algorithm!

Why again ? If not convinced , See the following recursion tree!

Recursive Fibonacci : Recursion Tree Example

Blue boxes are base cases. Growth of the tree is very fast

Notice repeated computations of the same value : Fib(1),FibFib(2),Fib(3)…



Example-3 Recursive Binary Search

```
RecbinSearch ( A, first, last, x)
  if ( first <= last )
    mid = (first + last) / 2;     // mid is int.
      if  x =A(mid )    // found it!
          return mid;
          else if  x < A(mid)   // must be in 1st half
                  return RecbinSearch( A, first, mid-1, x);
                else          // must be in 2nd half
                  return RecbinSearch(A, mid+1, last, x);
  return -1;  //Not found
```

- No loop! Recursive calls takes its place.

Analysis of Recursive Binary Search

- The recurrence is the same as Iterative algorithm

$T(N)$ = Number of comparisons with array elements

$T(1) = 1$

$T(N) = 1 + T(N / 2)$

……

$= \log_2 N + 1$

$= O(\log_2 N)$

The complexity is logarithmic.

But actual run time is bigger in recursive binary search.

Advantages of Recursion

- Recursion works the best when the algorithm and/or data structure that is used naturally supports recursion.
- **One such data structure is the tree structure another is the binary search algorithm.**
- Recursive solutions are clearer, simpler, shorter, and easier to understand than their non-recursive counterparts.
- Usually, recursion directly reflects the abstract solution strategy .
- From a practical software engineering point of view these are important benefits, reducing the cost of maintaining the software.

Stack Frames in Recursion

- A stack is used to remember parameters and local variables across recursive calls
- Each recursive invocation gets its own stack frame
- A stack frame contains storage for current call :
  - Local variables
  - Parameters
  - Return info (return address and return value)
  - Other bookkeeping info
- A new stack frame is pushed with each recursive call
- The stack frame is popped when the function returns

Limitations of Recursion

- In general, recursive algorithms run slower than their iterative counterparts. Why?
  - When a recursive call is made, it takes time to build a stackframe for the call and push it onto the system stack. This takes time.
  - Conversely, for every return , the stackframe must be popped from the stack and the local variables must be reset to their previous values – this also takes time.
- **Therefore, if the recursion is deep, say, factorial(100) or Fibonacci(20) we may run out of memory.**
- **Because of this, it is usually best to develop iterative algorithms when we are working with large numbers.**
- Be warned :

Your simple, elegant recursive algorithms may be inherently inefficient!